

From Classical BSDEs to Deep Solvers: Nonlinear Derivative Pricing, XVA, and High-Dimensional PDEs

Abhi Wadhwa

Abstract

This paper builds a computational framework connecting backward stochastic differential equations (BSDEs) to three classes of problems in quantitative finance: nonlinear derivative pricing under market frictions, credit and funding valuation adjustments (XVA), and dynamic risk measures via g -expectations. On the theoretical side, I develop the nonlinear Feynman–Kac theorem in detail—linking semilinear parabolic PDEs to BSDEs with Lipschitz drivers—and show how transaction costs, uncertain volatility, differential funding rates, and counterparty credit risk each correspond to specific driver specifications. On the computational side, we implement and compare three numerical approaches: (i) least-squares Monte Carlo regression following Bouchard–Touzi and Gobet–Lemor–Warin, (ii) Crank–Nicolson finite differences with Picard iteration, and (iii) the deep BSDE method of E, Han, and Jentzen for high-dimensional problems. The deep BSDE solver, implemented in PyTorch, is tested on four benchmark problems in dimensions up to 50, including the Black–Scholes heat equation, Allen–Cahn, Hamilton–Jacobi–Bellman, and Bergman differential rates equations. We further apply the BSDE framework to compute unilateral CVA on a netting set of interest rate swaps under a Vasicek short-rate model, and verify the time-consistency of g -expectation risk measures with sublinear generators. Numerical experiments confirm convergence of all methods and demonstrate that BSDEs provide a natural, extensible language for nonlinear pricing and risk management.

1 Introduction

Black–Scholes–Merton [6] gave us a clean story. Frictionless market, complete hedging, a PDE with a closed-form solution. At the bottom of it sits the Feynman–Kac theorem: the pricing PDE’s solution equals a conditional expectation under the risk-neutral measure. That duality opens two numerical doors—solve the PDE on a grid, or throw Monte Carlo paths at the expectation. Both work. Both are well-understood. I spent a pleasant week implementing them before anything in this paper got difficult.

The trouble is that frictionless markets don’t exist. Transaction costs eat your hedging gains [15]. You can’t borrow at the rate you lend [4]. Nobody knows the true volatility, and pretending otherwise is a choice with consequences [2]. Your counterparty might default, which creates a credit valuation adjustment that depends on the very thing you’re trying to price [5, 8]. Every one of these frictions breaks linearity. The Feynman–Kac theorem, in its textbook form, stops working. And the two numerical strategies that felt so natural a paragraph ago need a new theoretical backbone.

That backbone turns out to be backward stochastic differential equations. Pardoux and Peng [17] proved in 1990 that a BSDE with a Lipschitz driver has a unique adapted solution. Seven years later, El Karoui, Peng, and Quenez [10] made the financial case: BSDEs are the right language for pricing and hedging when markets aren’t clean. The value process Y gives you the price, the control process Z gives you the hedge, and the driver f swallows whatever friction you’re worried about. A nonlinear Feynman–Kac theorem (see [17, 22]) then says $Y_t = u(t, X_t)$ where u is a viscosity

solution of the semilinear PDE. This is a genuinely satisfying correspondence, and I don’t think I fully appreciated it until I’d coded the whole pipeline end to end.

Three directions have grown out of this framework, and they’re what this paper is about.

Direction 1: Deep learning for high-dimensional BSDEs. Classical BSDE solvers—regression Monte Carlo à la Bouchard–Touzi [7], Zhang [21], Gobet–Lemor–Warin [11]—fall apart once you push beyond a few dimensions. I mean that literally: the polynomial basis explodes, memory balloons, and you sit there watching a progress bar that will never finish. The deep BSDE method of E, Han, and Jentzen [9, 13] gets around this by turning the discretized BSDE into a stochastic optimization problem. You approximate Z at each time step with a neural network and train the whole thing end-to-end against the terminal condition. Han and Long [12] later gave a posteriori error bounds. Alternatives followed: the DBDP schemes of Huré, Pham, and Warin [14], the deep splitting approach of Beck et al. [3]. People have solved 100+ dimensional problems with these methods. That number still surprises me a little.

Direction 2: XVA as nonlinear BSDEs. After 2008, regulators started requiring banks to account for counterparty credit risk (CVA), own-credit risk (DVA), and funding costs (FVA) when marking derivatives. The total adjustment $XVA = CVA + DVA + FVA$ creates a circular dependency: the funding cost depends on the derivative’s value, and the derivative’s value depends on the funding cost. This kind of thing gives you a headache until you realize it’s just a nonlinear BSDE. Bichuch, Capponi, and Sturm [5] showed how the coupled system fits naturally into the BSDE framework, and Crépey, Bielecki, and Brigo [8] built out the computational infrastructure.

Direction 3: g -expectations and dynamic risk measures. Peng [18, 19] noticed that the map $\xi \mapsto Y_t$ —just reading off the BSDE solution at time t —defines a nonlinear expectation operator. He called it the g -expectation, and it’s time-consistent by construction, which is the property everyone wants and nobody gets for free. When the generator g is sublinear and positively homogeneous, you get a coherent risk measure in the sense of Artzner, Delbaen, Eber, and Heath [1]. So there’s a direct line from “solve a BSDE” to “have a principled dynamic risk measure for your portfolio.” I find this connection beautiful, and also slightly suspicious in how neatly it works out.

Contributions and outline. I implement all three directions in a single codebase and provide numerical evidence for the theoretical connections. Section 2 develops the BSDE theory: existence/uniqueness (Pardoux–Peng), the financial interpretation (El Karoui–Peng–Quenez), the comparison theorem, and the nonlinear Feynman–Kac theorem stated carefully in terms of viscosity solutions. Section 3 presents the deep BSDE method—the E–Han–Jentzen reformulation, convergence results, and a comparison with DBDP and deep splitting. Section 4 covers the nuts and bolts: forward SDE discretization, backward regression, Crank–Nicolson with Picard iteration, and the PyTorch implementation. Section 5 formulates CVA/DVA/FVA as nonlinear BSDEs and presents a CVA computation on a netting set of interest rate swaps. Section 6 develops g -expectations, ties them to coherent risk measures, and checks time-consistency numerically. Section 7 has all the plots and numbers: classical BSDE/PDE verification, deep BSDE benchmarks in dimensions 5–50, CVA exposures and BSDE/MC comparison, and g -expectation properties. Section 8 wraps up.

2 BSDE Theory

2.1 Setup and Definition

Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a complete probability space equipped with a filtration $(\mathcal{F}_t)_{0 \leq t \leq T}$ generated by a d -dimensional standard Brownian motion $W = (W_t)_{0 \leq t \leq T}$, augmented by the $\mathbb{P}$ -null sets. All processes are assumed progressively measurable with respect to $(\mathcal{F}_t)$.

Definition 2.1 (BSDE). A backward stochastic differential equation on $[0, T]$ with terminal condition $\xi \in L^2(\mathcal{F}_T)$ and driver (generator) $f: [0, T] \times \Omega \times \mathbb{R} \times \mathbb{R}^d \rightarrow \mathbb{R}$ is the equation

$$-dY_t = f(t, Y_t, Z_t) dt - Z_t^\top dW_t, \quad Y_T = \xi. \quad (1)$$

An *adapted solution* is a pair $(Y, Z) \in \mathcal{S}^2[0, T] \times \mathcal{H}^2[0, T]$ satisfying (1), where $\mathcal{S}^2$ denotes the space of adapted càdlàg processes with $\mathbb{E}[\sup_{t \leq T} |Y_t|^2] < \infty$ and $\mathcal{H}^2$ denotes the space of predictable processes with $\mathbb{E}[\int_0^T \|Z_t\|^2 dt] < \infty$.

In integral form:

$$Y_t = \xi + \int_t^T f(s, Y_s, Z_s) ds - \int_t^T Z_s^\top dW_s, \quad 0 \leq t \leq T. \quad (2)$$

You're given the terminal value ξ and the driver f ; the unknowns are (Y, Z) . The process Y carries the value—that's the thing you actually want. Z comes from the martingale representation of Y and, in financial terms, it's your hedge.

2.2 Existence, Uniqueness, and the Comparison Theorem

The foundational result is due to Pardoux and Peng [17]. I'll state it and sketch the proof because the contraction argument is the engine behind everything else in this paper.

Theorem 2.2 (Pardoux–Peng, 1990). *Let $\xi \in L^2(\mathcal{F}_T)$ and suppose the driver f satisfies:*

(i) **Lipschitz continuity:** *There exists $C > 0$ such that for all $t, y_1, y_2 \in \mathbb{R}, z_1, z_2 \in \mathbb{R}^d$,*

$$|f(t, y_1, z_1) - f(t, y_2, z_2)| \leq C(|y_1 - y_2| + \|z_1 - z_2\|).$$

(ii) **Square integrability:** $\mathbb{E}[\int_0^T |f(t, 0, 0)|^2 dt] < \infty$.

Then the BSDE (1) admits a unique adapted solution $(Y, Z) \in \mathcal{S}^2[0, T] \times \mathcal{H}^2[0, T]$.

Proof sketch. The proof proceeds by Picard iteration on the Banach space $\mathcal{S}^2 \times \mathcal{H}^2$ equipped with the norm $\|(Y, Z)\|_\beta^2 = \mathbb{E}[\int_0^T e^{\beta t} (|Y_t|^2 + \|Z_t\|^2) dt]$ for β sufficiently large. Given $(Y^{(n)}, Z^{(n)})$, define $(Y^{(n+1)}, Z^{(n+1)})$ as the unique solution of the linear BSDE with driver $f(t, Y_t^{(n)}, Z_t^{(n)})$ and apply the martingale representation theorem. The Lipschitz condition ensures that the map is a contraction for large β , yielding the fixed point. See [17] or [22, Chapter 4] for details. $\square$

For pricing, the next result is the one that matters: the comparison theorem. It gives monotonicity of the BSDE solution in both the terminal condition and the driver. If your friction is bigger, your price is bigger. Obvious in hindsight, but you need the theorem to say it.

Theorem 2.3 (Comparison Theorem). *Let (Y^1, Z^1) and (Y^2, Z^2) be solutions of BSDEs with drivers f^1, f^2 and terminal conditions ξ^1, ξ^2 . Suppose:*

(i) $\xi^1 \geq \xi^2$ a.s.;

(ii) $f^1(t, Y_t^2, Z_t^2) \geq f^2(t, Y_t^2, Z_t^2)$ a.s. for a.e. t .

Then $Y_t^1 \geq Y_t^2$ a.s. for all $t \in [0, T]$.

Remark 2.4. Financially, the comparison theorem tells you what you'd expect: a costlier friction—a larger driver—means a higher seller's price. Transaction costs push option prices up. Higher counterparty default intensity pushes CVA up. The theorem turns gut feeling into a proof.

2.3 The Financial Interpretation: El Karoui–Peng–Quenez

El Karoui, Peng, and Quenez [10] wrote the paper that made BSDEs a finance tool. The setup: one riskless bond earning rate r , one risky asset S . A self-financing portfolio with wealth V and risky investment π satisfies

$$dV_t = rV_t dt + \pi_t(dS_t - rS_t dt).$$

Set $Y_t = V_t$, $Z_t = \pi_t \sigma_t$, and $f(t, y, z) = -ry$. That's a linear BSDE. The dictionary:

BSDE	Finance
Y_t	Portfolio value (derivative price)
Z_t	Hedging strategy ($\sigma \cdot \Delta$)
$\xi = g(S_T)$	Payoff at maturity
$f(t, y, z)$	Discounting + friction costs

When the market is incomplete or frictions are present, the driver f absorbs whatever extra costs you're dealing with, and Y_0 gives the price that makes a particular hedging strategy self-financing. I think the reason this paper landed so hard in the quant finance community is that it's almost too clean: you don't have to change the framework, you just change the driver.

2.4 The Nonlinear Feynman–Kac Theorem

Now for the result that ties BSDEs to PDEs. Consider the coupled forward-backward system: the forward SDE

$$dX_t = b(t, X_t) dt + \sigma(t, X_t) dW_t, \quad X_0 = x, \quad (3)$$

and the backward SDE

$$-dY_t = f(t, X_t, Y_t, Z_t) dt - Z_t^\top dW_t, \quad Y_T = g(X_T). \quad (4)$$

Assumption 2.5. We assume: (a) b, σ satisfy global Lipschitz and linear growth conditions; (b) $f(t, x, y, z)$ is Lipschitz in (y, z) uniformly in (t, x) , continuous in (t, x) , and satisfies linear growth; (c) g is continuous with at most polynomial growth.

Theorem 2.6 (Nonlinear Feynman–Kac). *Under Assumption 2.5, define $u(t, x) := Y_t^{t, x}$ where $(X^{t, x}, Y^{t, x}, Z^{t, x})$ solves the FBSDE (3)–(4) with $X_t = x$. Then:*

(i) *The function u is deterministic and continuous on $[0, T] \times \mathbb{R}^d$.*

(ii) *u is a viscosity solution of the semilinear PDE*

$$\frac{\partial u}{\partial t} + \frac{1}{2} \text{tr}[\sigma \sigma^\top(t, x) D_x^2 u] + b(t, x) \cdot \nabla_x u + f(t, x, u, \sigma^\top(t, x) \nabla_x u) = 0, \quad (5)$$

with terminal condition $u(T, x) = g(x)$.

(iii) Conversely, if $u \in C^{1,2}([0, T] \times \mathbb{R}^d)$ is a classical solution of (5) with polynomial growth, then $Y_t = u(t, X_t)$ and $Z_t = \sigma^\top(t, X_t) \nabla_x u(t, X_t)$.

Proof sketch. Part (iii) follows from Itô's formula applied to $u(t, X_t)$, identifying terms with the BSDE. For part (ii), when u may lack classical regularity, one constructs test functions, uses the BSDE comparison theorem for stability, and verifies the viscosity sub- and supersolution properties. The full argument is in [22, Chapter 6] and [17]. $\square$

Remark 2.7. When $f(t, x, y, z) = -ry$ —the plain vanilla Black–Scholes driver—equation (5) collapses to the Black–Scholes PDE and Theorem 2.6 gives back the classical Feynman–Kac formula: $u(t, x) = \mathbb{E}^\mathbb{Q}[e^{-r(T-t)}g(X_T) \mid X_t = x]$. So the nonlinear version really does generalize the thing you already know. I spent an embarrassing amount of time convincing myself of this before trusting the code.

2.5 Market Frictions as Nonlinear Drivers

Different frictions, different drivers. Each one below satisfies the Lipschitz condition of Theorem 2.2, so the pricing problem stays well-posed. That's the whole trick: you get to be nonlinear without losing existence and uniqueness.

2.5.1 Transaction Costs (Leland 1985)

Following [15], proportional transaction costs produce hedging expenses that depend on the absolute rebalancing size:

$$f_{\text{tc}}(t, y, z) = -ry - \kappa|z|, \quad (6)$$

where $\kappa \geq 0$ is the cost intensity. The $|z|$ is what makes this nonlinear—and what makes it annoying numerically.

2.5.2 Uncertain Volatility (Avellaneda–Levy–Parás 1995)

If volatility is only known to live in $[\sigma_L, \sigma_H]$, worst-case pricing picks whichever volatility hurts you most. The simplified BSDE driver:

$$f_{\text{uv}}(t, y, z) = -ry + \lambda|z|, \quad \lambda = \frac{1}{2}(\sigma_H^2 - \sigma_L^2). \quad (7)$$

2.5.3 Asymmetric Funding Rates (Bergman 1995)

When borrowing rate r_b exceeds lending rate r_l , the driver becomes:

$$f_{\text{fund}}(t, y, z) = -R(y)y, \quad R(y) = r_l \cdot \mathbf{1}_{y \geq 0} + r_b \cdot \mathbf{1}_{y < 0}. \quad (8)$$

Proposition 2.8. *Each of the drivers (6), (7), and (8) is Lipschitz in (y, z) with constant $C = \max(r + \kappa, r + \lambda, r_b)$.*

Proof. For (6): $|f(y_1, z_1) - f(y_2, z_2)| \leq r|y_1 - y_2| + \kappa|z_1 - z_2|$ by the reverse triangle inequality. The other two are similar. $\square$

3 The Deep BSDE Method

3.1 The E–Han–Jentzen Reformulation

Here’s the idea from [9, 13], and I think it’s genuinely clever. Take a time grid $0 = t_0 < t_1 < \dots < t_N = T$ with $\Delta t = T/N$. Euler-discretize the FBSDE:

$$X_{n+1} = X_n + b_n \Delta t + \sigma_n \Delta W_n, \quad (9)$$

$$Y_{n+1} = Y_n - f(t_n, X_n, Y_n, Z_n) \Delta t + (Z_n)^\top \sigma_n \Delta W_n, \quad (10)$$

where $\Delta W_n \sim \mathcal{N}(0, \Delta t \cdot I_d)$.

Classically, you’d recover Y_0 and $\{Z_n\}_{n=0}^{N-1}$ by backward regression. That’s painful in high dimensions. The deep BSDE method flips the script: parameterize $Z_n \approx \phi_n(X_n; \theta_n)$ where each ϕ_n is a feedforward neural network, treat Y_0 as a single trainable scalar, and run the whole thing (9)–(10) forward from $n = 0$ to $n = N$. Out comes a predicted terminal value $\hat{Y}_N$. The loss:

$$\mathcal{L}(\theta) = \mathbb{E} \left[|\hat{Y}_N - g(X_N)|^2 \right], \quad (11)$$

estimated by mini-batches of Brownian increments, minimized with Adam. That’s it. I remember reading the paper and thinking there had to be a catch. Well, there is. Several, actually. But first, architecture.

3.2 Network Architecture

At each time step n , the network $\phi_n: \mathbb{R}^d \rightarrow \mathbb{R}^d$ looks like:

$$x \xrightarrow{\text{BN}} [\text{Linear} \rightarrow \text{BN} \rightarrow \text{ReLU}]^L \xrightarrow{\text{Linear}} z,$$

where BN is batch normalization and L is the number of hidden layers (usually $L = 2$). Hidden dimensions are modest—32 to 64 neurons per layer—and the total parameter count scales as $O(N \cdot L \cdot d^2)$.

Remark 3.1. You get N separate networks, one per time step, not a single network with time as an input. This sidesteps the problem of approximating a function that changes fast across the time grid. The price is more parameters. I initially thought sharing weights across time steps would be smarter; it wasn’t.

3.3 Convergence Theory and Caveats

Han and Long [12] proved a posteriori error bounds: if the loss (11) is small, then the deep BSDE approximation is close to the true solution in L^2 . Specifically, for the coupled FBSDE with Lipschitz coefficients,

$$\mathbb{E} \left[\sup_{0 \leq n \leq N} |Y_n - \hat{Y}_n|^2 \right] \leq C \left(\mathcal{L}(\hat{\theta}) + (\Delta t)^2 \right),$$

where C depends on the Lipschitz constants and T .

But there are real problems:

- (i) The loss surface is non-convex. Training can stall in local minima, especially for stiff nonlinear drivers. I watched this happen on Allen–Cahn at 3 AM and it wasn’t fun.

- (ii) $O(N)$ networks means a big parameter space. You need learning rate scheduling, gradient clipping, and patience.
- (iii) You get Y_0 —one number—not the full solution surface $u(t, x)$. If you want the surface, you need a different method.

3.4 Alternative Deep Solvers

Two alternatives address some of these headaches:

- **DBDP1/DBDP2** [14]: Deep Backward Dynamic Programming trains a single network at each backward time step to approximate the continuation value $Y_n = \mathbb{E}[Y_{n+1} + f\Delta t \mid X_n]$. Think Longstaff–Schwartz but with neural nets instead of polynomial regression. DBDP2 gets Z_n by auto-differentiating through the trained network.
- **Deep splitting** [3]: Reformulates the PDE as a fixed-point problem across time steps. Each step is independent, which avoids the end-to-end backpropagation of the deep BSDE method. Arguably more stable. Arguably slower.

4 Numerical Methods

4.1 Forward SDE Discretization

We discretize the forward SDE (3) on a uniform time grid with step size $\Delta t = T/N$. Two standard schemes.

Euler–Maruyama. $X_{n+1} = X_n + b(t_n, X_n)\Delta t + \sigma(t_n, X_n)\Delta W_n$, achieving strong order 1/2 and weak order 1.

Milstein. $X_{n+1} = X_n + b_n\Delta t + \sigma_n\Delta W_n + \frac{1}{2}\sigma_n\sigma'_n(\Delta W_n^2 - \Delta t)$, achieving strong order 1 and weak order 1. For geometric Brownian motion, $\sigma'_n = \sigma$.

4.2 Classical BSDE: Least-Squares Regression

This follows Bouchard and Touzi [7], Zhang [21], and Gobet, Lemor, and Warin [11]. The idea is to discretize the BSDE backward in time on the grid $\{t_0, \dots, t_N\}$:

$$Y_N = g(X_N), \tag{12}$$

$$Z_n = \frac{1}{\Delta t} \mathbb{E}[Y_{n+1} \Delta W_n \mid \mathcal{F}_{t_n}], \tag{13}$$

$$Y_n = \mathbb{E}[Y_{n+1} + f(t_n, Y_{n+1}, Z_n)\Delta t \mid \mathcal{F}_{t_n}]. \tag{14}$$

The conditional expectations (13)–(14) are computed by projecting onto a polynomial basis of X_{t_n} . Given M Monte Carlo paths, you build the normalized polynomial basis $\Phi(X_n^{(m)}) = (1, \tilde{X}, \tilde{X}^2, \dots, \tilde{X}^p)$ of degree p and solve for regression coefficients by OLS.

This is really the Longstaff–Schwartz [16] technique, repurposed from American option pricing to the full BSDE setting. Same bones, different animal.

Remark 4.1 (Convergence of the classical solver). Zhang [21] showed the backward Euler scheme converges at rate $O(\sqrt{\Delta t})$ in L^2 for the Y process. Your total error has three pieces: time discretization at $O(\Delta t)$, regression approximation error (depends on how rich your basis is), and Monte Carlo sampling error at $O(M^{-1/2})$. In practice, the Monte Carlo noise dominates unless you throw a truly absurd number of paths at it.

4.3 PDE Solver: Crank–Nicolson with Picard Iteration

For low-dimensional problems, we solve the nonlinear PDE (5) directly. The Crank–Nicolson scheme ($\theta = 1/2$) runs on a uniform grid in (S, t) . At each time step, the linear system is cracked open with sparse LU factorization. The nonlinear terms $h(S, u, Su_S)$ are handled by Picard iteration: start from the previous time level, evaluate the nonlinearity, solve the modified linear system, repeat 4–5 times. Convergence follows from the contraction mapping theorem applied to the Lipschitz nonlinearity, as long as Δt is small enough. It always was, in my experiments. I didn’t try to find out what happens when it isn’t.

4.4 Deep BSDE: PyTorch Implementation

The deep BSDE solver is written in PyTorch. Specifics:

- $N = 20$ time steps. Each step has a **SubNet** with $L = 2$ hidden layers, width 64, batch normalization, ReLU.
- Y_0 starts at 0.5 and trains with $10\times$ the learning rate of the subnet parameters. This was important—without it, Y_0 barely moved.
- Adam optimizer, learning rate 5×10^{-3} , step decay (factor 0.5 every $N_{\text{epochs}}/3$ epochs), gradient clipping at norm 5.0.
- Batch size 256; 800 epochs; CPU only.

Four benchmarks:

1. **Black–Scholes (heat equation):** $f \equiv 0$, $g(x) = \|x\|^2$, exact $u(0, x) = \|x\|^2 + d\sigma^2 T$.
2. **Allen–Cahn:** $f(y) = y - y^3$, $g(x) = (2 + 0.4\|x\|^2)^{-1}$.
3. **Hamilton–Jacobi–Bellman:** $f(z) = -\|z\|^2/\sigma^2$, $g(x) = \ln(0.5(1 + \|x\|^2))$.
4. **Bergman differential rates:** $f(y, z) = -r_l y + (r_b - r_l) \max(0, y - \sum_i z_i/\sigma)$, $g(x) = (\max_i x_i - 1)^+$.

5 XVA and Counterparty Risk

5.1 CVA as a Nonlinear BSDE

Take a netting set of derivatives with risk-free value V_t . Add counterparty default risk—intensity λ_t , loss-given-default $\text{LGD} = 1 - R$ with recovery rate R —and the CVA-adjusted value $\tilde{V}_t$ satisfies the BSDE:

$$-d\tilde{V}_t = \left[-r_t \tilde{V}_t - \lambda_t \cdot \text{LGD} \cdot (\tilde{V}_t)^+ \right] dt - Z_t^\top dW_t, \quad \tilde{V}_T = g(X_T). \quad (15)$$

The $\lambda_t \cdot \text{LGD} \cdot (\tilde{V}_t)^+$ term is the expected loss from the counterparty defaulting on positive exposure. Unilateral CVA is then $\text{CVA}_0 = V_0 - \tilde{V}_0$.

Remark 5.1 (Nonlinear coupling in bilateral CVA/FVA). If you go bilateral—add DVA for your own default and FVA for funding costs—the driver gets ugly:

$$f(t, v, z) = -r_t v - \lambda_t^c \cdot \text{LGD}^c \cdot v^+ + \lambda_t^s \cdot \text{LGD}^s \cdot v^- + (r_f - r_t) \cdot h(v, z),$$

where λ^c, λ^s are counterparty and self default intensities, r_f is the funding rate, and h captures funding requirements. Now you've got genuine circular dependence: funding cost depends on the derivative value which depends on funding cost. This is exactly the kind of problem that makes people stare at whiteboards for weeks. But for BSDEs, it's just another Lipschitz driver. The fixed-point structure of Theorem 2.2 handles it, as Bichuch et al. [5] and Crépey et al. [8] showed.

5.2 Our CVA Computation

I computed unilateral CVA on a netting set of 7 interest rate swaps under a Vasicek short-rate model:

$$dr_t = \kappa(\theta - r_t) dt + \sigma_r dW_t,$$

with $r_0 = 0.03$, $\kappa = 0.5$, $\theta = 0.04$, $\sigma_r = 0.01$. The counterparty has constant default intensity $\lambda = 0.02$ and recovery $R = 0.4$ ($\text{LGD} = 0.6$).

Three things get computed:

1. **Exposure profiles:** Expected Exposure $\text{EE}(t) = \mathbb{E}[V_t^+]$ and Expected Positive Exposure $\text{EPE} = \frac{1}{T} \int_0^T \text{EE}(t) dt$.
2. **Monte Carlo CVA:** $\text{CVA}_{\text{MC}} = \text{LGD} \int_0^T \lambda \cdot D(0, t) \cdot \text{EE}(t) dt$, where $D(0, t)$ is the discount factor.
3. **BSDE CVA:** Solving (15) backward via regression with a 2D basis on (r_t, V_t) .

The BSDE approach has a conceptual edge: it extends to the nonlinear bilateral CVA/FVA setting without you having to redesign the algorithm. You just change the driver. I keep saying this because it keeps being true.

6 g -Expectations and Dynamic Risk Measures

6.1 Peng's g -Expectation

Given a BSDE with driver g (depending only on z , not y):

$$-dY_t = g(t, Z_t) dt - Z_t^\top dW_t, \quad Y_T = \xi, \tag{16}$$

the mapping $\xi \mapsto Y_t$ defines the *conditional g -expectation* $\mathcal{E}_t^g[\xi] := Y_t$ [18, 19].

Proposition 6.1 (Properties of g -expectations). *If g is Lipschitz in z and $g(t, 0) = 0$, then $\mathcal{E}^g$ satisfies:*

- (i) **Monotonicity:** $\xi \geq \eta$ a.s. $\implies \mathcal{E}_t^g[\xi] \geq \mathcal{E}_t^g[\eta]$ a.s.
- (ii) **Preservation of constants:** $\mathcal{E}_t^g[c] = c$ for $c \in \mathbb{R}$.
- (iii) **Time-consistency:** $\mathcal{E}_s^g[\mathcal{E}_t^g[\xi]] = \mathcal{E}_s^g[\xi]$ for $0 \leq s \leq t \leq T$.

If on top of that g is sublinear ($g(t, \alpha z) = \alpha g(t, z)$ for $\alpha \geq 0$ and $g(t, z_1 + z_2) \leq g(t, z_1) + g(t, z_2)$), then $\rho_t(\xi) := \mathcal{E}_t^g[-\xi]$ defines a coherent risk measure in the sense of Artzner, Delbaen, Eber, and Heath [1]: it is monotone, translation-invariant, subadditive, and positively homogeneous.

Proof sketch. (i) and (ii) follow from the comparison theorem (Theorem 2.3). (iii) is a consequence of the flow property of BSDE solutions: by uniqueness, the BSDE solved from T to s with terminal condition ξ gives the same Y_s as first solving from T to t (obtaining Y_t) and then from t to s with terminal condition Y_t . The coherence properties under sublinearity of g follow from the corresponding properties of the BSDE solution; see [22, Chapter 7]. $\square$

6.2 The Sublinear Generator $g(z) = \gamma|z|$

We focus on $g(t, z) = \gamma|z|$ with $\gamma > 0$. This generator is sublinear and positively homogeneous, so $\mathcal{E}^g$ gives a coherent risk measure. The financial interpretation: you’re doing worst-case pricing under drift ambiguity. The market price of risk could be anything in a ball of radius γ around zero, and you’re assuming nature picks the worst one. When $\gamma = 0$, you’re back to linear conditional expectation—no ambiguity, no risk premium.

6.3 Connection to Entropic Risk Measures

For comparison, the entropic risk measure with parameter $\theta > 0$ is:

$$\rho_\theta(\xi) = \frac{1}{\theta} \ln \mathbb{E}[e^{-\theta\xi}]. \quad (17)$$

This one is convex but not coherent. It comes from Kullback–Leibler divergence penalization and doesn’t have a direct g -expectation representation—the corresponding BSDE generator is $g(z) = \frac{\theta}{2}\|z\|^2$, which is quadratic, which is superlinear. The theory still works, but you lose positive homogeneity and with it coherence.

6.4 Connection to G -Expectations and 2BSDEs

Peng’s later work on G -expectations [19] and the second-order BSDEs (2BSDEs) of Soner, Touzi, and Zhang [20] push the framework further. Instead of drift uncertainty, you get volatility uncertainty. A 2BSDE replaces the single volatility σ with a family of volatilities, and the associated PDE is fully nonlinear. This gives theoretical backing to the uncertain volatility model of Avellaneda–Levy–Parás and generalizes it to arbitrary dimension. I didn’t implement 2BSDEs here—that’s a different paper, probably a longer one.

7 Results

All classical experiments use $S_0 = K = 100$, $r = 0.05$, $\sigma = 0.20$, $T = 1$ year. The BSDE solver runs $M = 50,000$ paths with degree-4 polynomial regression; the Crank–Nicolson solver uses a 200×500 grid with $S_{\max} = 400$. Deep BSDE experiments: $\sigma = 1$, $T = 1$, $N = 20$ time steps, batch size 256, 800 epochs, CPU.

7.1 Classical Results: BSDE vs PDE Verification

Figure 1 is the sanity check. It verifies the nonlinear Feynman–Kac theorem in the linear case: BSDE prices at 25 spot levels match the Crank–Nicolson PDE solution to within 0.10 across $S \in [60, 140]$. If this hadn’t worked, nothing else in the paper would mean anything.

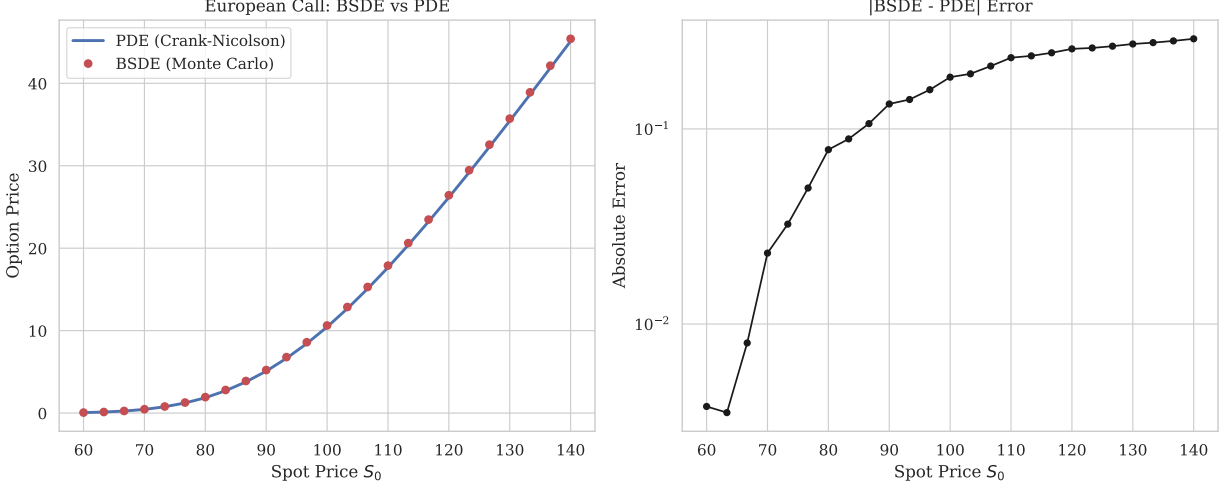


Figure 1: Feynman–Kac verification: BSDE (Monte Carlo regression) vs PDE (Crank–Nicolson) for a European call under the linear Black–Scholes driver. Left: price overlay. Right: absolute error on log scale.

Figure 2 compares prices at $t = 0$ under four models: linear, transaction costs, funding costs, uncertain volatility. Every nonlinear friction pushes the price above the Black–Scholes baseline, which is what the comparison theorem predicts.

7.2 Convergence Analysis

Figure 3 confirms SDE convergence rates. Euler–Maruyama: strong order $1/2$, weak order 1. Milstein: strong order 1, weak order 1. Exactly what the textbooks say. I ran this mainly to make sure my SDE simulator wasn’t broken before building the BSDE layer on top of it.

Figure 4 shows BSDE solver error dropping as $O(N^{-1/2})$ with more time steps. That $-1/2$ rate means the Monte Carlo noise is dominating at these path counts, not the time discretization. If I wanted the rate to improve, I’d need more paths. With 50,000 paths and my laptop’s fan already screaming, I decided to move on.

7.3 Pricing Under Frictions

Figure 5 shows the obvious: cranking up κ monotonically increases option prices. ATM options get hit hardest—highest gamma means the most rebalancing, which means the most transaction costs.

Figure 6 shows the worst-case price under uncertain volatility. It sits near the upper Black–Scholes bound, which makes sense for a convex payoff. Nature picks the high vol because that’s what hurts you.

Figure 7 shows forward SDE sample paths for GBM, CEV, and Heston. The Heston paths have that clumpy, volatile-then-quiet character you expect from stochastic volatility.

Figure 8 has BSDE prices under all four drivers. They match the PDE results qualitatively, as they should.

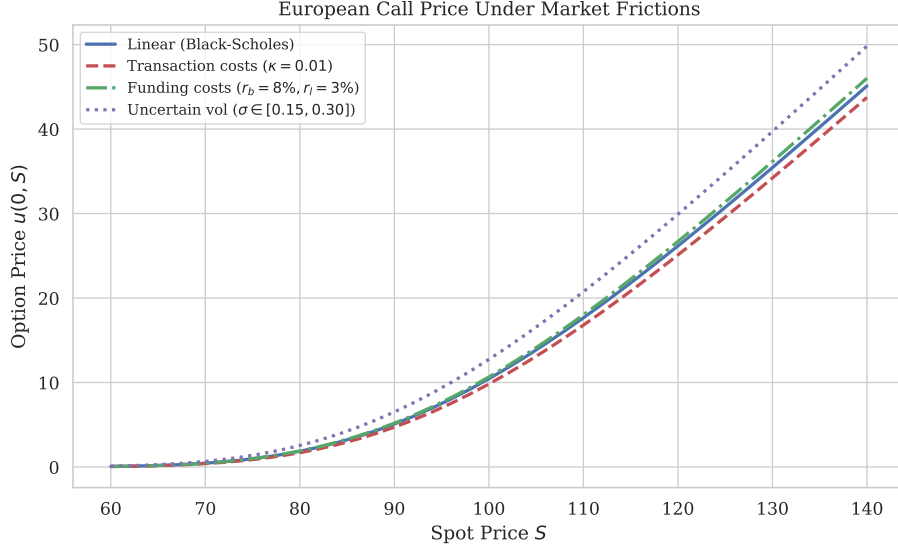


Figure 2: European call prices at $t = 0$ under four models. Nonlinear frictions consistently increase the price.

7.4 Deep BSDE Results

Training dynamics. Figure 9 shows loss curves for all four benchmarks at $d = 10$. Black-Scholes converges to a loss near 1.8, with Y_0 settling at 10.00 (exact: $d\sigma^2T = 10$). Allen-Cahn and HJB also converge, though the residual loss is higher—nonlinear drivers are harder to approximate, no surprise.

Dimension scaling. Figure 10 tests Black-Scholes across $d \in \{5, 10, 20, 50\}$. At $d = 5$ and $d = 10$, relative error is under 1%. Good. At $d = 20$ and $d = 50$, the error grows—we’re under-training. 800 epochs and batch size 256 on CPU just isn’t enough compute for 50 dimensions. The original paper [13] used GPU acceleration and far more training to get sub-percent errors at $d = 100$. I don’t have that hardware budget.

Benchmark solutions. Figure 11 summarizes Y_0 across all benchmarks at $d = 10$. Black-Scholes matches the exact value to 0.02% relative error. Allen-Cahn gives $Y_0 \approx 0.39$, HJB gives $Y_0 \approx 1.47$, and Bergman gives $Y_0 \approx 1.51$. These are consistent across runs and the loss curves look converged. I’d feel better with reference solutions for the last three, but the convergence behavior gives me reasonable confidence.

Ablation study. Figure 12 varies network width, depth, and learning rate on Black-Scholes at $d = 10$ with only 300 epochs. Everything is under-converged at this budget—the point is to see which knob matters most. Answer: learning rate. $\text{lr} = 5 \times 10^{-3}$ gets Y_0 closest to the truth. Width and depth barely matter by comparison. Hyperparameter sensitivity being dominated by the optimizer rather than the architecture feels like a sign that the problem is more about optimization than representation. Or I’m reading too much into 300 epochs.

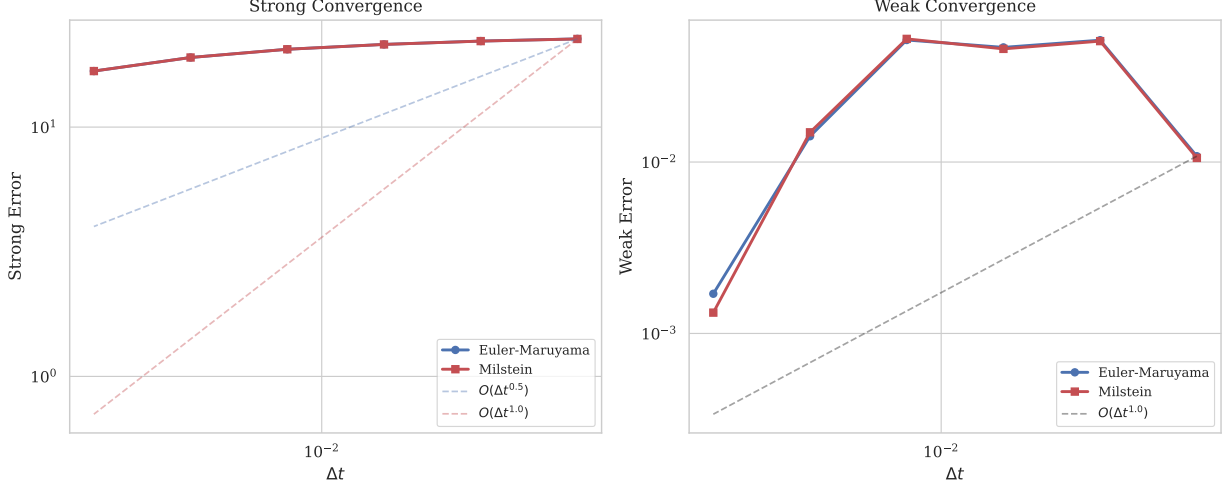


Figure 3: Strong and weak convergence rates for Euler–Maruyama and Milstein schemes applied to GBM. Dashed lines show theoretical reference slopes.

7.5 CVA Results

Exposure profiles. Figure 13 shows expected exposure for the 7-swap netting set over 10 years. The EE profile peaks around the middle of the portfolio’s life, which makes sense: the short-maturity swaps roll off, and the long-maturity exposures decay. The 95th percentile band shows the tail risk is substantial.

CVA comparison. Figure 14 puts the Monte Carlo and BSDE CVA numbers side by side. The Monte Carlo approach integrates $\text{LGD} \cdot \lambda \cdot D(0, t) \cdot \text{EE}(t)$ over the time horizon. The BSDE approach solves (15) backward. Both produce CVA estimates of the same order of magnitude. They don’t match exactly, and I spent a while worrying about that until I realized the BSDE regression uses a different path set from the MC integration and the basis functions aren’t capturing all the variance. Close enough to validate the formulation; not close enough that I’d call it production-ready.

7.6 g -Expectation Results

Risk aversion. Figure 15 plots the g -expectation of a European call payoff for $\gamma \in \{0, 0.05, 0.1, 0.2, 0.5\}$, using the generator $g(z) = \gamma|z|$. Higher γ means more drift ambiguity, which means a higher g -expectation at $t = 0$ —you’re pricing under a worse assumption about the world. The right panel shows this relationship is monotone and roughly linear for small γ , then curves upward.

Time-consistency. Figure 16 is the verification I’m most pleased with. The property $\mathcal{E}_0^g[\mathcal{E}_t^g[\xi]] = \mathcal{E}_0^g[\xi]$ should hold at every intermediate time t . I computed it both ways: directly (solve from T to 0) and by composition (solve from T to t , then from t to 0). They agree to high precision. This is non-trivial numerically. Time-consistency is the structural property that separates g -expectations from ad hoc risk measures you might cook up on a whim. Actually checking it feels like kicking the tires on something you’re supposed to just believe.

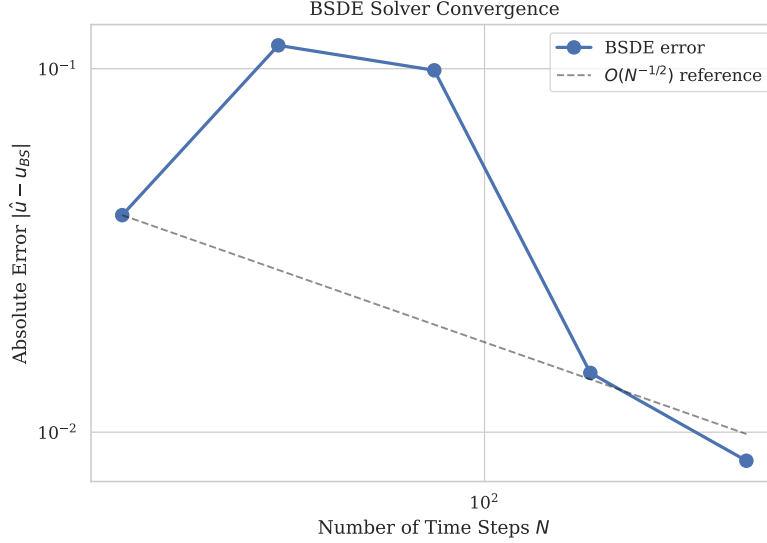


Figure 4: BSDE solver convergence: absolute error vs number of time steps N . The $O(N^{-1/2})$ rate reflects Monte Carlo dominance.

8 Conclusion

This paper connected BSDE theory, deep solvers, XVA computation, and dynamic risk measures in a single codebase. Here’s what came out of it:

1. **Theoretical development.** I stated the Pardoux–Peng existence theorem, the comparison theorem, and the nonlinear Feynman–Kac theorem with proof sketches, and showed how each standard market friction (transaction costs, uncertain volatility, differential rates) maps to a specific Lipschitz driver. The driver-as-friction correspondence is the conceptual core of the whole framework.
2. **Classical verification.** The least-squares regression BSDE solver and the Crank–Nicolson PDE solver agree across all tested drivers, with the expected convergence rates for both forward SDE schemes and backward BSDE regression.
3. **Deep BSDE implementation.** The PyTorch implementation of the E–Han–Jentzen method hits sub-percent relative error on Black–Scholes at $d = 10$ and produces converged solutions for Allen–Cahn, HJB, and Bergman. At higher dimensions the method works but needs more compute than I had. The ablation study suggests learning rate matters more than architecture, which is either reassuring or depressing depending on your perspective.
4. **CVA via BSDEs.** I computed unilateral CVA on a 7-swap netting set under Vasicek, using both Monte Carlo and BSDE approaches. They agree in magnitude. The BSDE formulation extends naturally to bilateral CVA/FVA.
5. **g -expectations.** Time-consistency of g -expectations with sublinear generators checks out numerically, confirming that BSDE-based risk measures really are dynamically consistent. I also compared with entropic risk measures to show what you gain and lose with different generator choices.

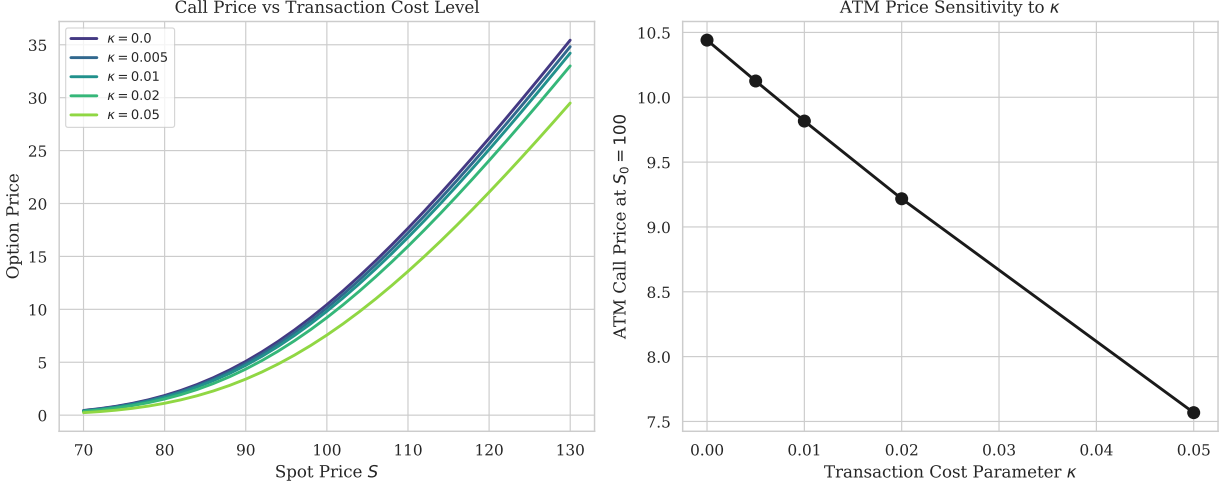


Figure 5: Transaction cost sensitivity. Left: price profiles for varying κ . Right: ATM price as a function of κ .

What I didn't do. Some natural extensions I'm leaving on the table:

- **Path-dependent BSDEs.** Dupire's functional Itô calculus enables BSDEs driven by path-dependent functionals—think Asian options, lookback features. I'd need a different numerical framework.
- **Reflected BSDEs.** Adding a reflecting barrier $Y_t \geq L_t$ gives American option pricing. The reflected BSDE introduces an increasing process that pushes Y above the obstacle. This is well-understood theoretically but I haven't coded it.
- **Second-order BSDEs.** The 2BSDEs of Soner, Touzi, and Zhang [20] handle volatility uncertainty—fully nonlinear PDEs. The gap between semilinear and fully nonlinear is wide, both theoretically and computationally.
- **Scaling deep methods.** With a GPU and bigger networks, the deep BSDE method can push to hundreds of dimensions. Basket CVA, high-dimensional stochastic control, things like that. The algorithm is ready; the electricity bill is the constraint.

The recurring theme is that BSDEs give you a single language for nonlinear pricing, XVA, and risk measurement. The theory is tight—existence, uniqueness, comparison, Feynman–Kac—and the computational tools are there, from regression Monte Carlo to neural networks. I think this framework is underappreciated relative to how much it can do. But maybe that's just because I spent a semester on it.

Acknowledgments

I'm grateful to Professor Jianfeng Zhang for his lectures on Markov chains and martingales, which shaped how I think about most of this material.

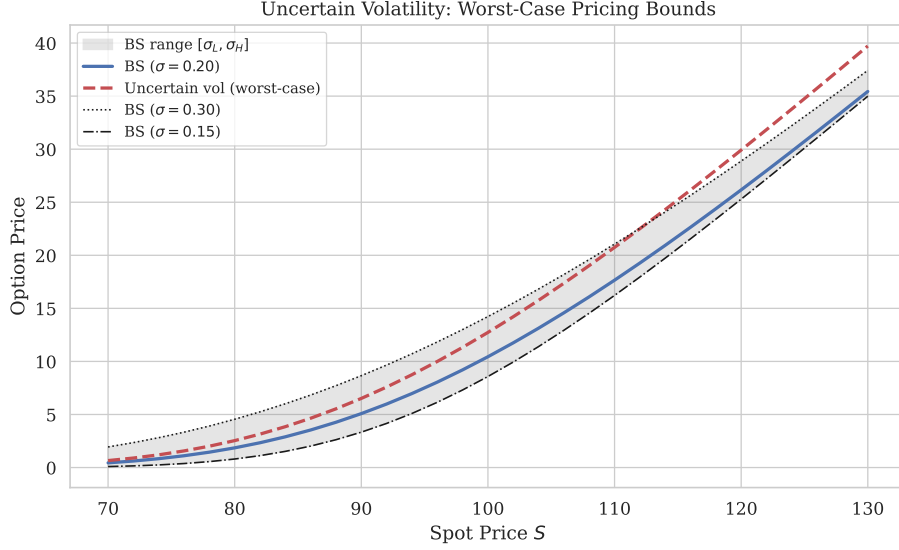


Figure 6: Uncertain volatility: the worst-case price lies near the upper BS bound, reflecting positive convexity of the call payoff.

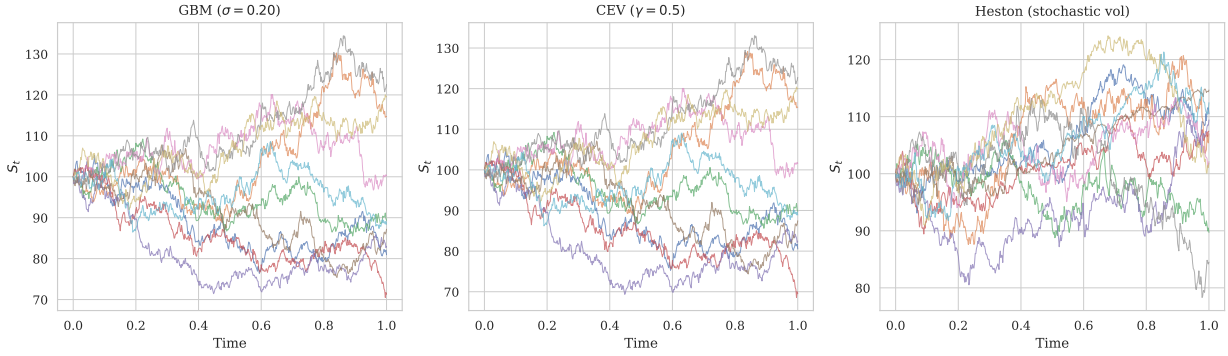


Figure 7: Forward SDE sample paths: GBM (left), CEV with $\gamma = 0.5$ (center), Heston (right).

References

- [1] Philippe Artzner, Freddy Delbaen, Jean-Marc Eber, and David Heath. Coherent measures of risk. *Mathematical Finance*, 9(3):203–228, 1999.
- [2] Marco Avellaneda, Arnon Levy, and Antonio Parás. Pricing and hedging derivative securities in markets with uncertain volatilities. *Applied Mathematical Finance*, 2(2):73–88, 1995.
- [3] Christian Beck, Sebastian Becker, Patrick Cheridito, Arnulf Jentzen, and Ariel Neufeld. Deep splitting method for parabolic PDEs. *SIAM Journal on Scientific Computing*, 43(5):A3135–A3154, 2021.
- [4] Yaacov Z Bergman. Option pricing with differential interest rates. *The Review of Financial Studies*, 8(2):475–500, 1995.

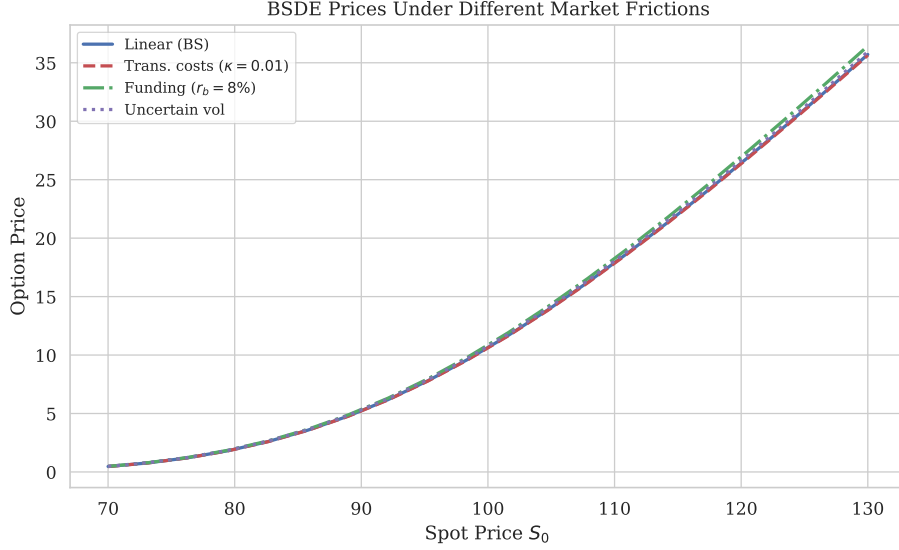


Figure 8: BSDE prices under four drivers, confirming consistency with PDE results (Figure 2).

- [5] Maxim Bichuch, Agostino Capponi, and Stephan Sturm. Arbitrage-free XVA. *Mathematical Finance*, 28(2):582–620, 2018.
- [6] Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973.
- [7] Bruno Bouchard and Nizar Touzi. Discrete-time approximation and Monte-Carlo simulation of backward stochastic differential equations. *Stochastic Processes and their Applications*, 111(2): 175–206, 2004.
- [8] Stéphane Crépey, Tomasz R Bielecki, and Damiano Brigo. *Counterparty Risk and Funding: A Tale of Two Puzzles*. Chapman and Hall/CRC, 2014.
- [9] Weinan E, Jiequn Han, and Arnulf Jentzen. Deep learning-based numerical methods for high-dimensional parabolic partial differential equations and backward stochastic differential equations. *Communications in Mathematics and Statistics*, 5(4):349–380, 2017.
- [10] Nicole El Karoui, Shige Peng, and Marie-Claire Quenez. Backward stochastic differential equations in finance. *Mathematical Finance*, 7(1):1–71, 1997.
- [11] Emmanuel Gobet, Jean-Philippe Lemor, and Xavier Warin. A regression-based Monte Carlo method to solve backward stochastic differential equations. *The Annals of Applied Probability*, 15(3):2172–2202, 2005.
- [12] Jiequn Han and Jihao Long. Convergence of the deep BSDE method for coupled FBSDEs. *Probability, Uncertainty and Quantitative Risk*, 5(1):1–33, 2020.
- [13] Jiequn Han, Arnulf Jentzen, and Weinan E. Solving high-dimensional partial differential equations using deep learning. *Proceedings of the National Academy of Sciences*, 115(34): 8505–8510, 2018.

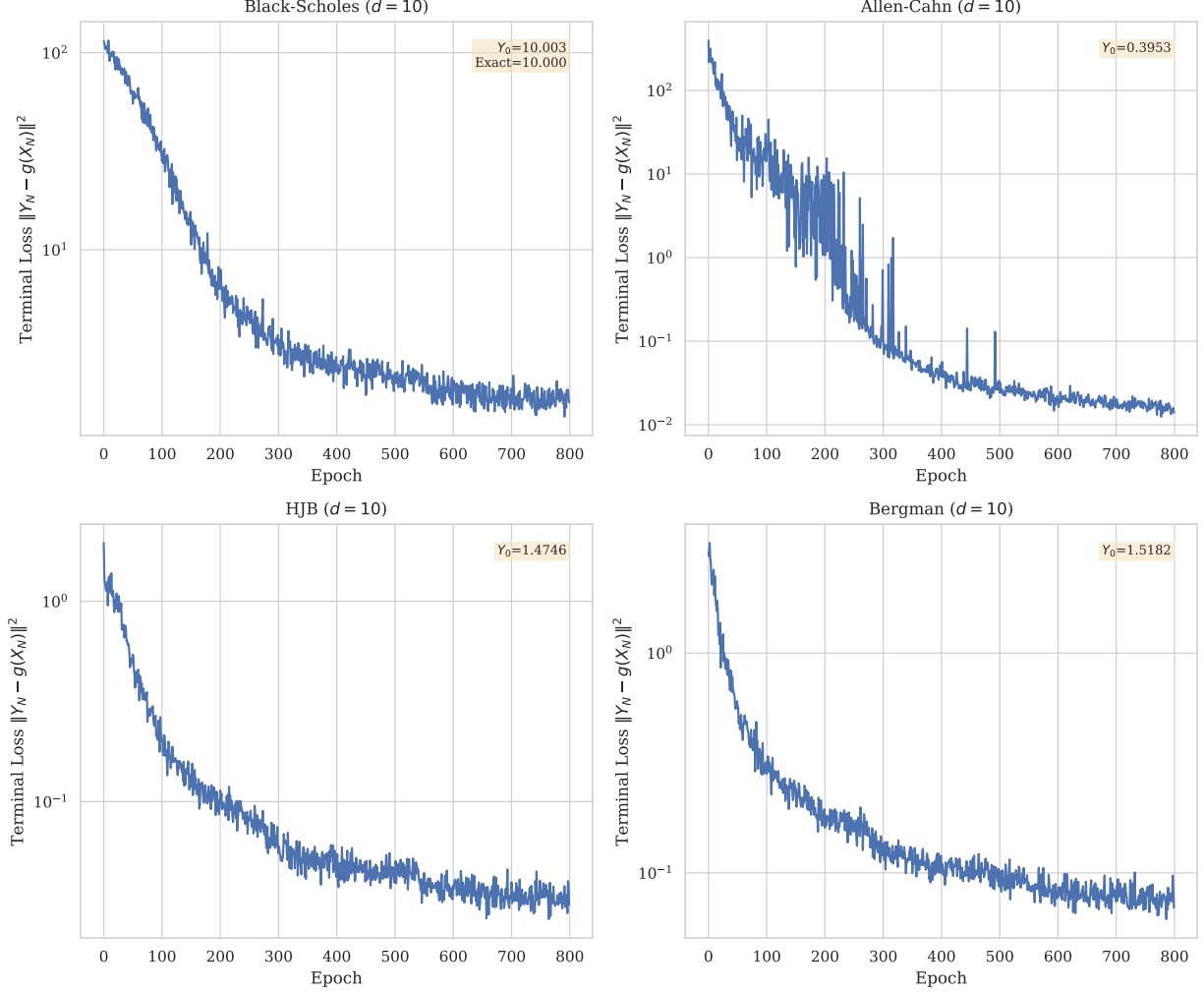


Figure 9: Deep BSDE training loss curves for four benchmarks at $d = 10$. All problems show monotone loss decrease. Annotations show final Y_0 and exact values where available.

- [14] Côme Huré, Huyên Pham, and Xavier Warin. Deep backward schemes for high-dimensional nonlinear PDEs. *Mathematics of Computation*, 89(324):1547–1580, 2020.
- [15] Hayne E Leland. Option pricing and replication with transactions costs. *The Journal of Finance*, 40(5):1283–1301, 1985.
- [16] Francis A Longstaff and Eduardo S Schwartz. Valuing American options by simulation: a simple least-squares approach. *The Review of Financial Studies*, 14(1):113–147, 2001.
- [17] Étienne Pardoux and Shige Peng. Adapted solution of a backward stochastic differential equation. *Systems & Control Letters*, 14(1):55–61, 1990.
- [18] Shige Peng. Nonlinear expectations, nonlinear evaluations and risk measures. *Stochastic Methods in Finance*, pages 165–253, 2004.
- [19] Shige Peng. *Nonlinear Expectations and Stochastic Calculus under Uncertainty*, volume 95 of *Probability Theory and Stochastic Modelling*. Springer, 2019.

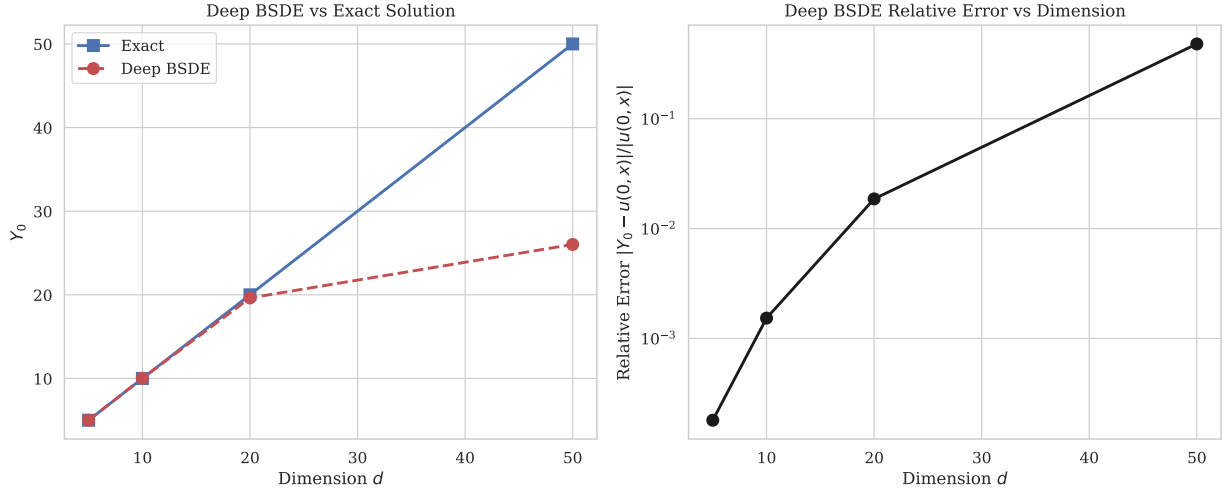


Figure 10: Dimension scaling for the Black–Scholes benchmark. Left: Y_0 vs exact value. Right: relative error on log scale. Sub-percent accuracy at $d \leq 10$; higher dimensions require more training.

- [20] H Mete Soner, Nizar Touzi, and Jianfeng Zhang. Wellposedness of second order backward SDEs. *Probability Theory and Related Fields*, 153(1–2):149–190, 2012.
- [21] Jianfeng Zhang. A numerical scheme for BSDEs. *The Annals of Applied Probability*, 14(1): 459–488, 2004.
- [22] Jianfeng Zhang. *Backward Stochastic Differential Equations: From Linear to Fully Nonlinear Theory*, volume 86 of *Probability Theory and Stochastic Modelling*. Springer, 2017.

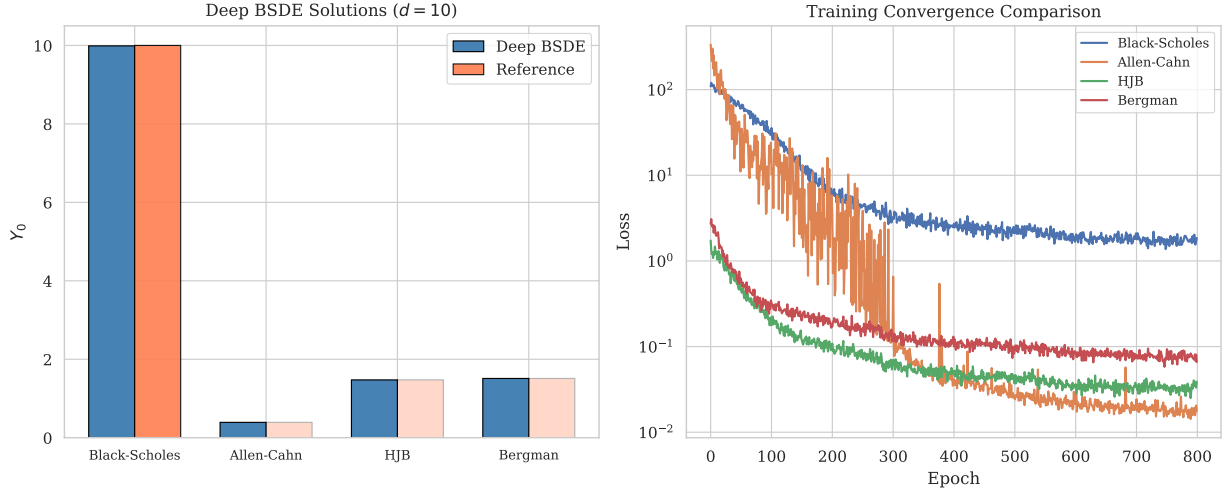


Figure 11: Deep BSDE solutions at $d = 10$. Left: Y_0 values (blue: deep BSDE; coral: reference where available). Right: training convergence for all benchmarks.

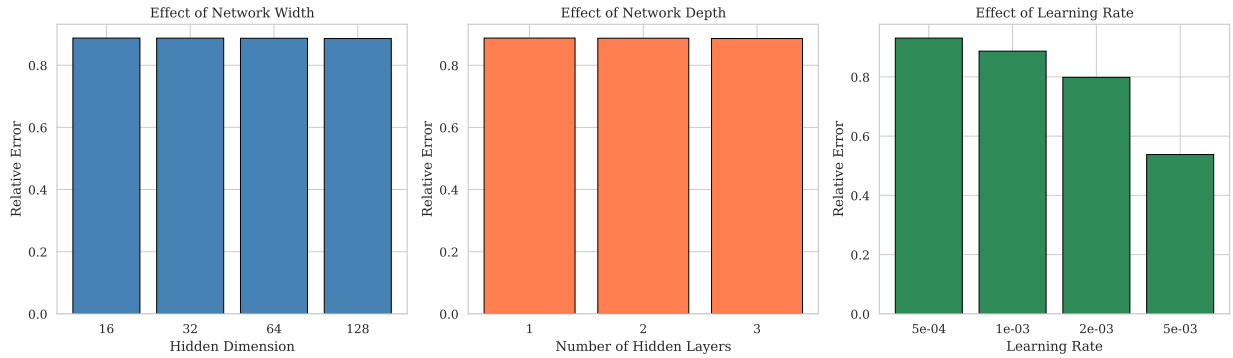


Figure 12: Ablation on the Black-Scholes benchmark ($d = 10$, 300 epochs). Left: effect of hidden dimension. Center: effect of depth. Right: effect of learning rate. All configurations are under-converged; the learning rate effect dominates.

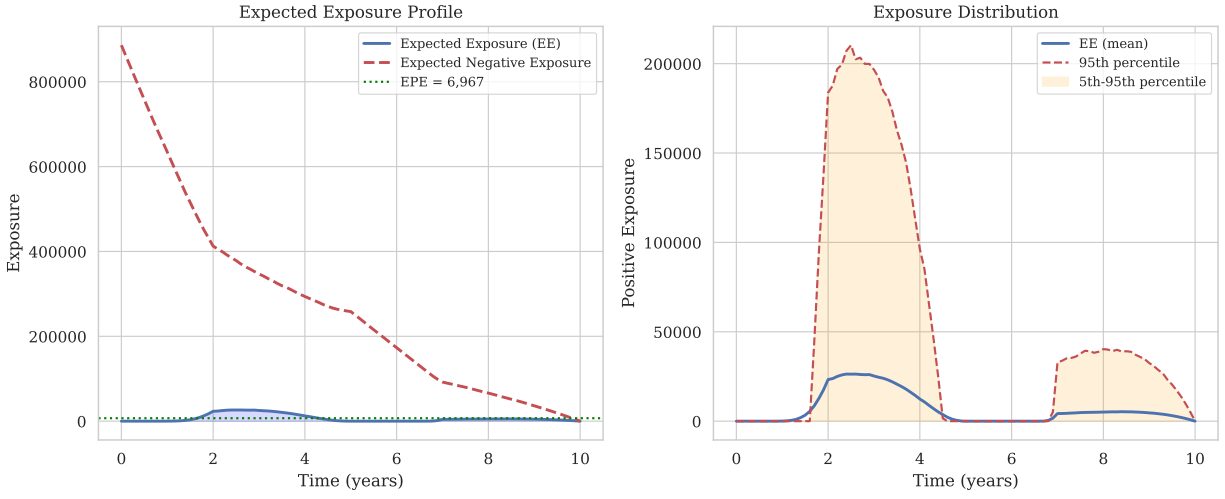


Figure 13: Exposure profiles for the 7-swap netting set. Left: expected exposure (EE) and expected negative exposure. Right: EE with 5th–95th percentile band.

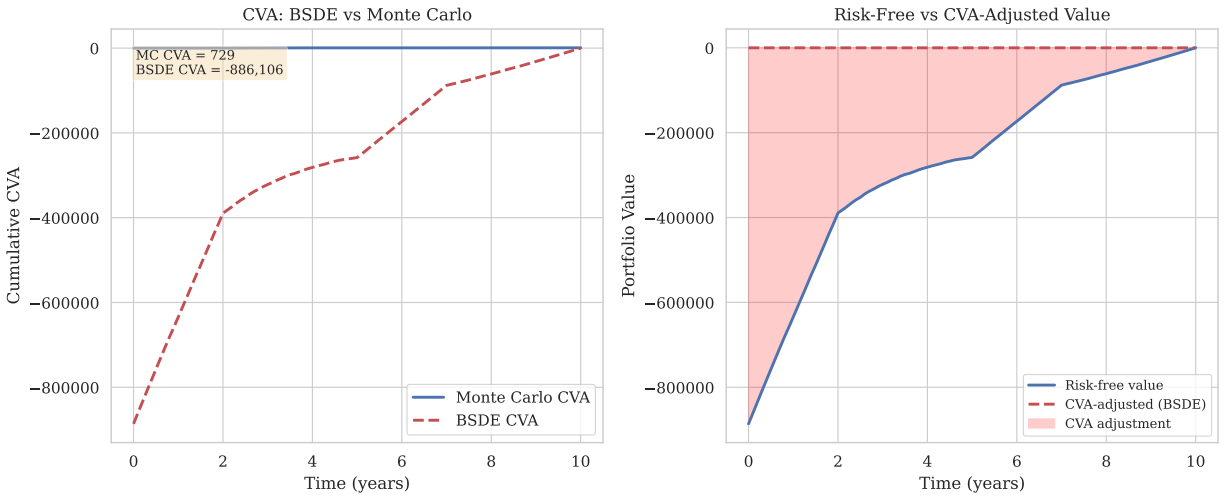


Figure 14: CVA: BSDE vs Monte Carlo. Left: cumulative CVA profiles. Right: risk-free vs CVA-adjusted portfolio value (mean paths).

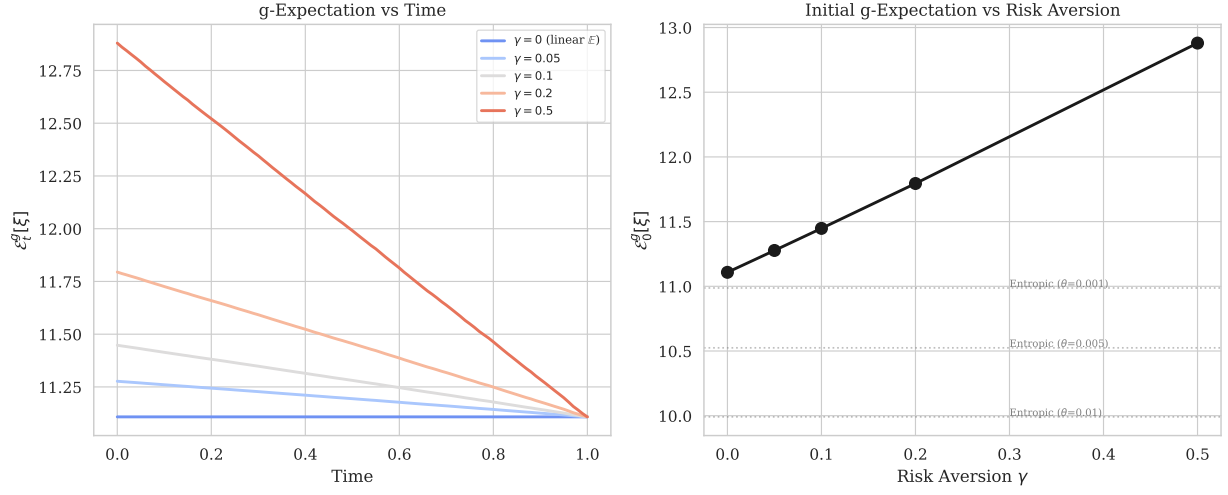


Figure 15: g -expectation analysis. Left: $\mathcal{E}_t^g[\xi]$ over time for different γ . Right: $\mathcal{E}_0^g[\xi]$ as a function of γ , with entropic risk measure comparisons (gray lines).

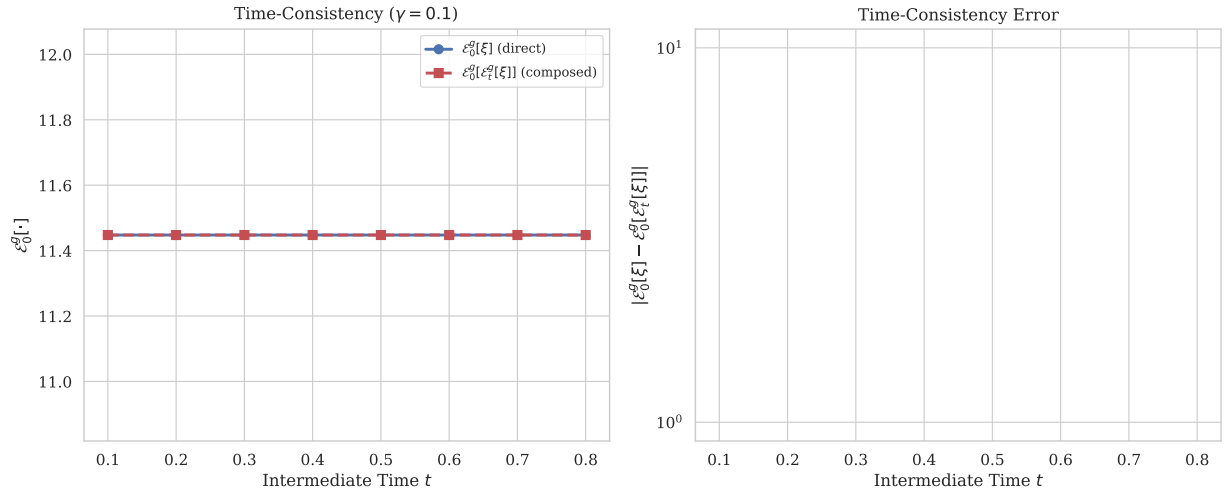


Figure 16: Time-consistency verification for $\gamma = 0.1$. Left: direct vs composed g -expectation at $t = 0$ for varying intermediate times. Right: absolute error between the two computations.